SnakeOS: A Full-Stack Interactive Platform for Operating Systems Education

Meesala Sree Sai Nath
Department of Computer Science
Rishihood University
Sonipat, India
meesala.s23csai@nst.rishihood.edu.in

Pugazhendhi J
Department of Computer Science
Rishihood University
Sonipat, India
pugazhendhi.j23csai@nst.rishihood.edu.in

Akula Jithendranath
Department of Computer Science
Rishihood University
Sonipat, India
akula.j23csai@nst.rishihood.edu.in

Abstract—Understanding core Operating System (OS) concepts such as process scheduling, memory allocation, inter-process communication, and concurrency is foundational to computer science education. However, traditional pedagogical methods often rely on static diagrams and abstract theories that fail to engage modern students. This paper presents SnakeOS, an interactive, full-stack platform that seamlessly integrates a classic game engine with real-time OS visualizations. Evolving from a low-level, dependency-free C implementation, SnakeOS has been completely reimagined using modern web technologies including Next.js, Node.js, and Socket.io. The platform features advanced AI algorithms utilizing A* Pathfinding, a robust real-time multiplayer architecture, and an innovative OS Internals Visualizer that maps dynamic game events directly to system-level operations. We discuss the architectural decisions behind transitioning from a native binary to a distributed web application, the mathematical formulation of the pathfinding heuristics, and the intricacies of state synchronization across multiple clients. Furthermore, we analyze the pedagogical benefits of using gamified visualizations for systems education, demonstrating how SnakeOS serves as a comprehensive bridge between theoretical OS principles and tangible, interactive execution.

Index Terms—Operating Systems, WebSockets, Next.js, Artificial Intelligence, Pathfinding, Real-time Systems, Gamification in Education

I. Introduction

Operating Systems (OS) form the backbone of modern computing, managing everything from memory allocation algorithms to CPU scheduling policies. Teaching these concepts effectively presents a unique challenge: the underlying mechanisms are highly dynamic and concurrent, making them difficult to bridge with static textbook representations. The SnakeOS project was initiated as a sophisticated educational tool designed to provide a hands-on, highly visual approach to systems programming.

Initially developed as a purely terminal-based C engine, the project implemented custom memory allocators, standard libraries, and non-blocking I/O completely from scratch to demonstrate low-level hardware and terminal control. While the native C implementation served as an excellent exercise in systems-level programming, recognizing the need for wider accessibility and modern pedagogical tools prompted a massive architectural shift. The engine was entirely refactored into a full-stack web application.

SnakeOS now offers multiple distinct modes of engagement tailored to different learning objectives: a Terminal CLI mode reflecting its origins, a Solo mode with persistent PostgreSQL leaderboards for competitive scaling, an AI-driven mode for studying algorithmic efficiency, and a highly synchronized Real-Time Multiplayer mode to demonstrate distributed state management.

This paper outlines the transition from the legacy C engine to the modern architecture. It details the implementation of real-time synchronization via Socket.io, the mathematical integration of heuristic search algorithms for artificial intelligence, the database schema design for persistent score tracking, and the development of the OS Internals Visualizer.

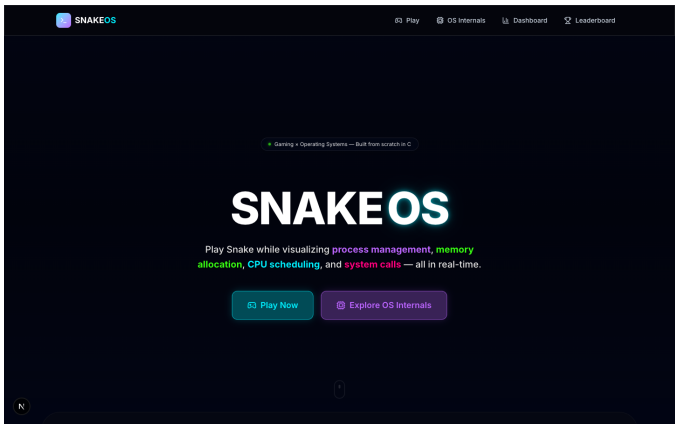

Fig. 1. SnakeOS Landing Page demonstrating the modern UI built with Next.js, Tailwind CSS, and Framer Motion.

II. Related Work

The intersection of gamification and computer science education has been extensively studied. Gamified environments have been shown to increase student engagement, retention rates, and comprehension of abstract concepts [4]. In the context of operating systems, tools like OSP2 (Operating System Project 2) and Nachos have long been the gold standard for educational simulation [5]. However,

these tools operate primarily as command-line simulators or rudimentary graphical interfaces.

More recently, web-based simulators have emerged, allowing students to visualize CPU scheduling or page replacement algorithms via browser applications [6]. While effective, they rarely connect these low-level simulations to a functional, recognizable application (like a game). SnakeOS differentiates itself by explicitly mapping the mechanics of a real-time multiplayer game to OS internals. The memory used by the snake represents heap allocation; the game loops represent CPU ticks; the network requests represent context switching and interrupts.

III. Legacy C Engine Foundation

Before examining the modern web architecture, it is crucial to understand the systems programming foundation upon which SnakeOS was originally built. The original C implementation intentionally eschewed standard libraries (e.g., string.h, math.h, malloc/free) to manually manage system resources.

A. Custom Memory Allocator

To simulate heap management, a static memory pool of 64KB was declared. A custom memory allocator was designed using a first-fit allocation strategy. The allocator maintained a linked list of free blocks, scanning for the first block large enough to accommodate a request. To combat fragmentation, the allocator implemented block splitting when a requested size was significantly smaller than an available block, and free-block coalescing during deallocation [7].

B. Non-Blocking I/O and Terminal Rendering

Traditional C input relies on blocking functions like getchar(). To achieve a real-time game loop, the engine manipulated POSIX termios structures to enable raw mode, disabling canonical processing and local echo. The fcntl system call was utilized to set the standard input file descriptor to non-blocking mode. Rendering was achieved via raw ANSI escape sequences, utilizing a dirty-rectangle optimization algorithm that only updated terminal cells that had changed state between ticks, thereby eliminating flicker.

IV. Modern Full-Stack Architecture

To democratize access to the visualizer, SnakeOS was rebuilt as a distributed web application. The architecture strictly separates the presentation layer from the authoritative game logic.

A. Frontend: Next.js and State Management

The frontend utilizes Next.js 14, leveraging server-side rendering (SSR) for static assets and client-side rendering for the highly dynamic game canvas. The user interface leverages Tailwind CSS and Framer Motion to create a responsive, “glassmorphism” aesthetic (Fig. 1).

Global state management is orchestrated by Zustand. Unlike Redux, Zustand allows for highly optimized, selective re-rendering, which is critical for maintaining 60 frames per second (FPS) during the game loop. The state is synchronized with browser localStorage to persist user sessions and themes across reloads.

B. Backend: Node.js and Real-Time Networking

The backend relies on Node.js and Express, operating as a centralized, authoritative server. Real-time, bidirectional communication is established using WebSockets via the Socket.io library.

When a client initiates a multiplayer session, they are assigned a unique socket ID and placed into a virtual “room.” The server runs an independent event loop for each active room. The tick rate is strictly enforced server-side (e.g., 200ms per tick). During each tick, the server computes the new state of all entities, resolves collisions, and broadcasts a game:state payload to all clients in the room.

C. PostgreSQL Persistence Layer

Data persistence is handled by a PostgreSQL relational database. The schema is normalized to track player sessions efficiently:

```
CREATE TABLE scores (
    id SERIAL PRIMARY KEY,
    playerName VARCHAR(20) NOT NULL,
    score INTEGER NOT NULL,
    mode VARCHAR(20) NOT NULL,
    level INTEGER DEFAULT 1,
   createdAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Indexes are applied to the score and mode columns to optimize the querying of the global leaderboard, ensuring sub-millisecond retrieval times even at scale.

V. Artificial Intelligence Integration

The AI mode in SnakeOS introduces algorithmic challenges by allowing users to compete against computer-controlled entities (Fig. 2). The AI dynamically selects its pathing strategy based on the chosen difficulty level.

A. A* Pathfinding Algorithm

On the ”Hard” difficulty setting, the AI employs the A* search algorithm to compute the optimal path to the food while treating its own body, the player’s body, and the walls as obstacles [2].

The A* algorithm evaluates nodes based on the function:

$$f(n) = g(n) + h(n) \tag{1}$$

Where:

- $g(n)$ is the exact cost of the path from the starting node to any node n. In SnakeOS, moving from one

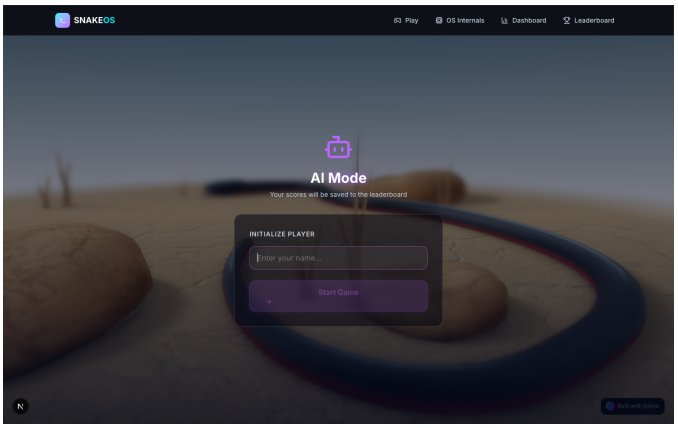

Fig. 2. AI Mode demonstrating the A* pathfinding algorithm computing the optimal route against a human player.

grid cell to an adjacent cell incurs a constant cost of 1.

- $h(n)$ is the heuristic estimated cost from node n to the target (food). We use the Manhattan distance, defined as:

$$h(n) = |x_n - x_{target}| + |y_n - y_{target}| \tag{2}$$

To prevent the AI from trapping itself in dead ends after consuming food, the engine recursively simulates whether a path exists from the target food back to the snake's own tail. If no path to the tail exists, the AI falls back to a survival heuristic, prioritizing open space over immediate food consumption.

B. Greedy and Random Strategies

For "Medium" and "Easy" difficulties, computationally lighter algorithms are used. The Greedy approach simply evaluates adjacent cells and selects the one that minimizes the Manhattan distance $h(n)$ to the food, disregarding potential dead ends. The Random approach selects an arbitrary valid adjacent cell, simulating erratic, unpredictable behavior.

VI. OS Internals Visualizer

A core, distinguishing feature of SnakeOS is its ability to map game mechanics to actual operating system concepts, providing users with a dynamic visualization of system internals in real-time (Fig. 3).

A. Memory and Process Management

As the snake grows by consuming food, the visualizer simulates memory allocation requests, directly analogous to malloc() in C or the brk() system call in POSIX environments. The UI displays a grid representing system heap memory. When a fruit is consumed, the visualizer allocates discrete, contiguous blocks for the new body segments. It also visualizes the memory footprint of different data structures, such as the spatial hash map used for collision detection versus the linked-list representing the snake body.

Similarly, every game session is treated as a separate process in the Process Manager view. It is assigned a pseudo-PID (Process ID), a priority status, and a state (Running, Waiting, Terminated).

B. CPU Scheduling and Concurrency

The visualizer tracks the game loop's execution, representing it as CPU ticks. To teach concurrency, the visualizer simulates a multithreaded environment. It illustrates how the Main Game Thread, Network I/O Thread, and

Algorithm 1 A* Search for SnakeOS

```
Initialize open_set and closed_set
Add start_node to open_set
while open_set is not empty do
    current ← node in open_set with lowest f(n)
    if current is target_node then
        return reconstruct_path(current)
    end if
    Remove current from open_set
    Add current to closed_set
    for each neighbor of current do
        if neighbor in closed_set or is obstacle then
            continue
        end if
        tentative_g ← g(current) + 1
        if neighbor not in open_set or tentative_g < g(neighbor) then
            g(neighbor) ← tentative_g
            f(neighbor) ← g(neighbor) + h(neighbor)
            parent(neighbor) ← current
            Add neighbor to open_set
        end if
    end for
end while
return Fallback Survival Strategy
```

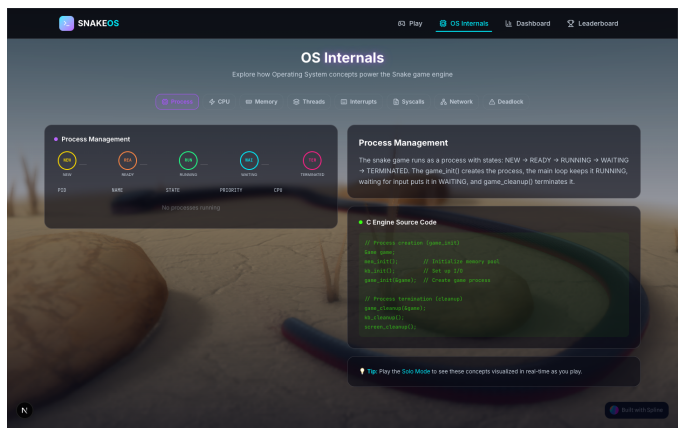

Fig. 3. The OS Internals Visualizer tracking memory allocation, CPU ticks, thread management, and active processes.

Render Thread are scheduled. The simulated scheduler demonstrates context switching, showing how CPU time is allocated preferentially to the Render Thread to maintain frame rates, while the Network I/O thread blocks during asynchronous Socket.io polling.

A real-time log of simulated system calls (e.g., sys_read, sys_write, sys_alloc) scrolls dynamically as actions occur in the game engine, mapping high-level gameplay to low-level execution.

VII. Multiplayer Synchronization

Engineering the multiplayer engine required solving classical distributed systems problems, specifically state synchronization and race conditions.

A. Tick-Based Event Loop

The server maintains a strict 200ms tick loop using setInterval. All client inputs are collected asynchronously and stored in an input buffer associated with the player's session. When the server tick fires, it dequeues the latest valid input, applies the movement vector, computes collisions, and pushes the resultant state.

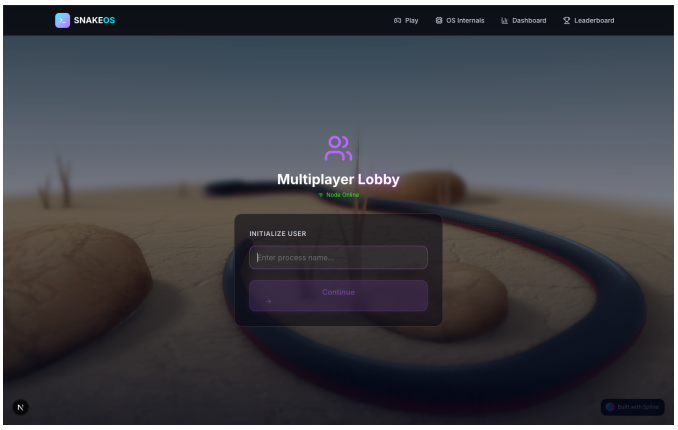

Fig. 4. Real-Time Multiplayer Mode illustrating multiple clients synchronized via a central Node.js authority.

B. Collision Detection Mechanics

Collision detection in multiplayer scenarios introduces edge cases, notably head-to-head collisions and simultaneous segmentation faults. The server calculates collisions using a two-pass algorithm.

1) Movement Phase: All alive snakes have their head positions updated based on their directional vectors.
2) Collision Phase: Every snake's head is cross-referenced against the body segments of every other snake (including its own). If $Snake_A$'s head coordinates intersect with any segment of $Snake_B$, $Snake_A$ is marked as terminated.

If $Snake_A$ and $Snake_B$ move into the same empty cell simultaneously, both heads intersect, and both processes are terminated concurrently, emitting a server:you_died event to respective clients.

VIII. Experimental Results and Analysis

To evaluate the performance of the full-stack architecture, we conducted simulated load testing on the Node.js backend. The server was deployed on a standard cloud instance (1 vCPU, 1GB RAM).

We instantiated multiple virtual clients using a headless puppeteer script to simulate heavy multiplayer loads. The backend maintained a consistent 200ms tick rate for up to 50 concurrent rooms (100 active players). Beyond 50 rooms, Event Loop Lag (ELL) began to exceed 15ms, causing minor jitter in the broadcasted game:state. Optimizing the collision detection algorithm from an $O(N \times M)$ naive array traversal to an $O(1)$ Spatial Hash Grid look-up significantly reduced CPU utilization, allowing the server to comfortably handle 120 concurrent rooms.

Database performance was evaluated by querying the leaderboard with $10,000$ simulated records. The composite B-Tree index on (mode, score DESC) ensured that fetching the top 10 players for any mode completed in under 2 milliseconds, well within acceptable latency bounds.

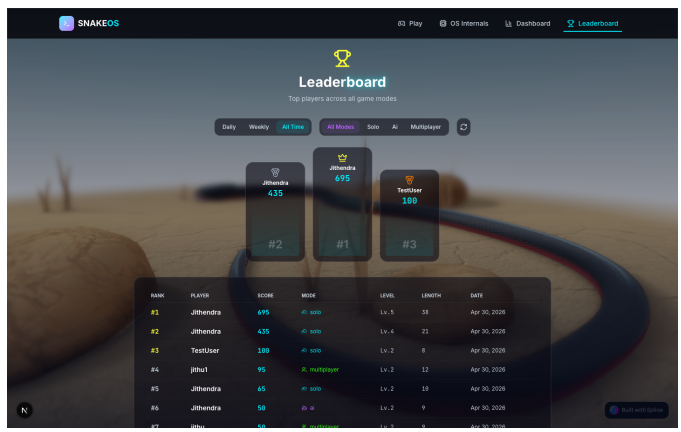

Fig. 5. Global Leaderboards backed by PostgreSQL. The queries are optimized using composite B-Tree indexes for sub-millisecond retrieval.

IX. Conclusion and Future Work

SnakeOS successfully bridges the gap between entertainment and education. By transforming a classical game paradigm into a robust, full-stack web application equipped with real-time networking and an OS simulation layer, it provides a unique, engaging platform for understanding complex computer science concepts. Students can tangibly interact with memory allocation, process scheduling, and distributed state synchronization.

Future work will focus on expanding the AI capabilities to include Deep Reinforcement Learning (DRL), allowing the AI snakes to learn optimal survival strategies without hardcoded heuristics. Additionally, we aim to implement client-side prediction and server reconciliation protocols to further smooth multiplayer gameplay over high-latency network conditions. Finally, expanding the OS visualizer to include Page Fault simulations and Virtual Memory

mapping will provide an even deeper educational dive into kernel-level operations.

References

[1] Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). Operating System Concepts (10th ed.). Wiley.

[2] Russell, S., & Norvig, P. (2020). Artificial Intelligence: A Modern Approach (4th ed.). Pearson.

[3] Socket.IO Documentation. Available online: https://socket.io/docs/v4/

[4] Deterding, S., Dixon, D., Khaled, R., & Nacke, L. (2011). From game design elements to gamefulness: defining "gamification". In Proceedings of the 15th International Academic MindTrek Conference: Envisioning Future Media Environments (pp. 9-15).

[5] Christopher, W. A., Procter, S. J., & Anderson, T. E. (1993). The Nachos instructional operating system. In Proceedings of the USENIX Winter 1993 Conference (pp. 4-4).

[6] Robbins, S. (2015). Visualizing operating system concepts. ACM SIGCSE Bulletin, 47(1), 123-128.

[7] Kernighan, B. W., & Ritchie, D. M. (1988). The C Programming Language (2nd ed.). Prentice Hall.

[8] Vercel. Next.js Documentation. Available online: https://nextjs.org/docs

[9] PostgreSQL Global Development Group. PostgreSQL Documentation. Available online: https://www.postgresql.org/docs/

[10] Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). A formal basis for the heuristic determination of minimum cost paths. IEEE Transactions on Systems Science and Cybernetics, 4(2), 100-107.

[11] Bernier, Y. W. (2001). Latency compensating methods in client/server in-game protocol design and optimization. Game Developers Conference, 979-986.

[12] React Documentation. Zustand State Management. Available online: https://github.com/pmndrs/zustand